

RAG Pipeline Overview — Lab 04

(Pandoogle: The Pandas Help Tool)

Completion requirements

RAG Pipeline Overview — Lab 04 (Pandoogle: The Pandas Help Tool)

This summarizes the retrieval-augmented generation (RAG) process used in **Lab 04**. Our goal is to let the model retrieve the most relevant *Pandas* documentation entries and use them to help answer user questions.

1. Corpus Preparation

For this lab, each **entry** in the JSON corpus corresponds to a single Pandas symbol. Each entry includes:

- `symbol`
- `signature`
- `doc`

These three fields are **concatenated** (with headings or separators) into a single text string and treated as **one document**.

We then embed the entire document rather than separate chunks. This keeps context together for each Pandas function or class.

Example document structure

```
### Symbol
pandas.DataFrame.head

### Signature
DataFrame.head(n=5)

### Documentation
Return the first n rows...
```

Embedding details

- Model: **Qwen Embedding Model** (chosen for its **32 K token limit**)
- Number of documents: **N ≈ 1055**
- Embedding dimension (*M*): depends on model size

- `qwen2:8b` → 4096 D
- `qwen2:0.6b` → smaller dimension

After embedding, you have a matrix **E** of shape $(N \times M)$:

- **N** = number of symbols (≈ 1055)
- **M** = embedding dimension (e.g., 4096)

2. User Query and Embedding

When the user enters a query (e.g.,

“How do I find the most common value in a column?”),

we embed that query text using the **same embedding model**, producing a vector **q** of shape $(M \times 1)$.

3. Retrieval (Semantic Search)

We compute **cosine similarity** between the query vector **q** and each row of **E**.

Mathematically:

```
scores = E @ q      # (N × M) @ (M × 1) → (N × 1)
```

Each element of `scores` represents how similar that symbol’s documentation is to the query.

Then:

- Sort or rank by similarity.
- Retrieve the top *K* results (typically 5 – 10).
- Optionally filter by a **minimum similarity threshold**.

Example code idea:

```
scores = E @ q
top_k = np.argsort(scores.ravel())[-K:][::-1]
```

4. Prompt Construction

The **retrieved documentation snippets** (from the top *K* matches) are concatenated with the **user’s query** to form a single, context-rich prompt.

Example prompt:

Use the following documentation snippets to answer the user's question.

```
--- Documentation ---
```

```
1. pandas.Series.value_counts()  
2. pandas.DataFrame.mode()  
3. pandas.value_counts()
```

```
--- User Question ---
```

```
I have a DataFrame column 'tip'. I want to know the most common tip and how  
many people tipped that amount.
```

5. Generation

Send the composed prompt to a **generation model** (for example, `gemma3:12b` via Ollama).

The generation model uses the retrieved documentation as its *grounding context* to produce an accurate, Pandas-aware response.

6. Loop

The system then repeats this cycle:

```
User query → Embed → Retrieve → Generate → Display → Wait for next query
```

Key Points to Remember

Concept	Symbol	Shape	Description
Embedding matrix	E	$(N \times M)$	Embeddings for all 1055 Pandas symbols
Query vector	q	$(M \times 1)$	Embedded user question
Similarity scores	E @ q	$(N \times 1)$	One cosine-like score per symbol
Retrieval	top K	—	Highest similarity scores
Generation	—	—	LLM produces grounded answer

Why Qwen?

The **Qwen embedding model** supports long input lengths (up to **32 K tokens**), which allows embedding of full documentation entries without splitting them into smaller chunks. This simplifies preprocessing and ensures that context (signature + docstring) stays together.

In Summary

1. **Embed the corpus once** → store as matrix E.
2. **Embed user queries** → vector q.
3. **Retrieve** the most relevant docs using cosine similarity.
4. **Augment** the prompt with those docs.
5. **Generate** a response grounded in actual Pandas documentation.

That's the retrieval-augmented generation pipeline behind **Lab 04: Pandoogle**.

Last modified: Monday, October 6, 2025, 11:16 AM